



university of
groningen

INTRODUCTION TO OPTIMIZATION

Computational Exercise II: Report

Authors:

Briana- Mihaela Balea

Elisaveta Bobrova

Student number:

S5310423

S5215250

Introduction

In this report we solve the Total Variation problem

$$\min_{X \in \mathbb{R}^{256 \times 256}} \left\{ \frac{\lambda}{2} \|R(X) - X_{\text{blur}}\|_2^2 + \|\text{grad}(X)\|_1 \right\}$$

using the Primal-Dual Method and ADMM in order to deblur an image. For the latter, the problem can be rewritten as

$$\min_{X \in \mathbb{R}^{256 \times 256}, Y \in \mathbb{R}^{2 \times 256 \times 256}} \left\{ \frac{\lambda}{2} \|R(X) - X_{\text{blur}}\|_2^2 + \|Y\|_1 \right\} \quad \text{subject to} \quad \text{grad}(X) = Y.$$

Prior to running the algorithms a few computations are required, the results of which we briefly discuss here. A more detailed description can be found in Appendix B. In what follows, we denote $\text{grad} := G$ to avoid confusing notation.

For the **Primal-Dual Method** we have $f(x) = \frac{\lambda}{2} \|Rx - b\|^2$, $g(Gx) = \|Gx\|_1$ and $h(x) = 0$. It immediately follows that $\tau\sigma \|G\|^2 \leq 1$, where $\|G\| \leq 2\sqrt{2}$ was given. The algorithm is as follows

$$\begin{cases} x_{k+1} = \text{prox}_{\tau f}(x_k - \tau G^T y_k) \\ y_{k+1} = \text{prox}_{\sigma g^*}(y_k + \sigma G(2x_{k+1} - x_k)). \end{cases}$$

Using *Moreau's Identity* and $f^*(z) = \frac{1}{2\lambda} \|z\|^2 + b \cdot z$ from Homework 3 we find that

$$\text{prox}_{\tau f}(v) = v - \tau \cdot \frac{\tau\lambda}{1 + \tau\lambda} (v - \tau^{-1}b) \quad \text{and} \quad \text{prox}_{\sigma g^*}(v)_i = \begin{cases} v_i & \text{if } v_i \in [-1, 1] \\ \text{sign}(v_i) & \text{otherwise} \end{cases}.$$

For **ADMM**, we have $f(x) = \frac{\lambda}{2} \|Rx - b\|^2$ and $g(y) = \|y\|_1$. After some manipulations, the algorithm becomes

$$\begin{cases} x_{k+1} = \text{argmin} \left\{ \frac{\lambda}{2} \|Rx - b\|^2 + \frac{\rho}{2} \|Gx - y_k + \frac{z_k}{\rho}\|^2 \right\} \\ y_{k+1} = \text{argmin} \left\{ \|y\|_1 + \frac{\rho}{2} \|y - Gx_{k+1} - \frac{z_k}{\rho}\|^2 \right\} \\ z_{k+1} = z_k + \rho(Gx_{k+1} - y_{k+1}) \end{cases} \iff \begin{cases} x_{k+1} = (\lambda R^T R + \rho G^T G)^{-1} (\lambda R^T b + G^T (\rho y_k - z_k)) \\ y_{k+1} = \text{sign} \left(Gx_{k+1} + \frac{z_k}{\rho} \right) \max \left(\left| Gx_{k+1} + \frac{z_k}{\rho} \right| - \frac{1}{\rho}, 0 \right) \\ z_{k+1} = z_k + \rho(Gx_{k+1} - y_{k+1}). \end{cases}$$

Figures

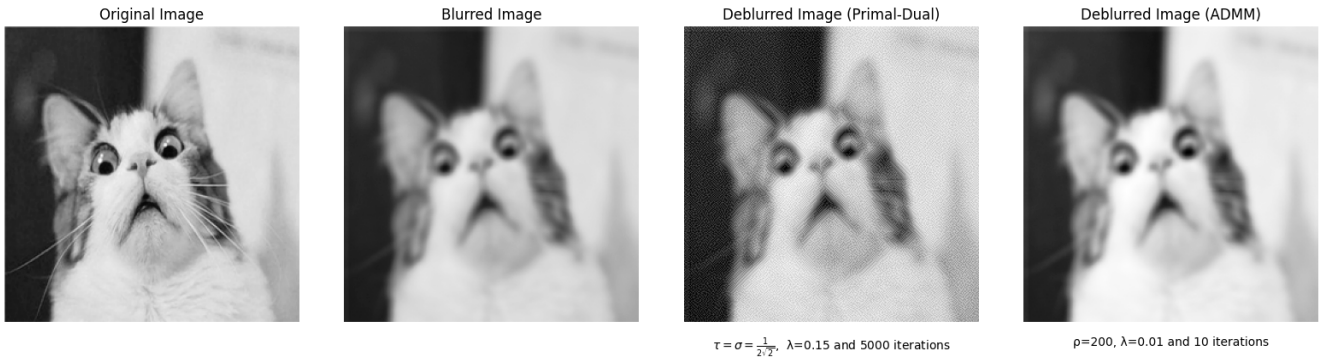


Figure 1: Comparison of the original image, the blurred image and the deblurred image, for both methods.

The values for parameters τ , σ and ρ were chosen via trial and error. In particular, for the Primal-Dual Method, the condition on the product of τ and σ was taken into consideration, that is, $\tau\sigma \leq \frac{1}{8}$.

A Plots

A.1 Primal-Dual

The following plots show the Primal-Dual Method being applied to the blurred image, using different values for λ and different number of iterations.

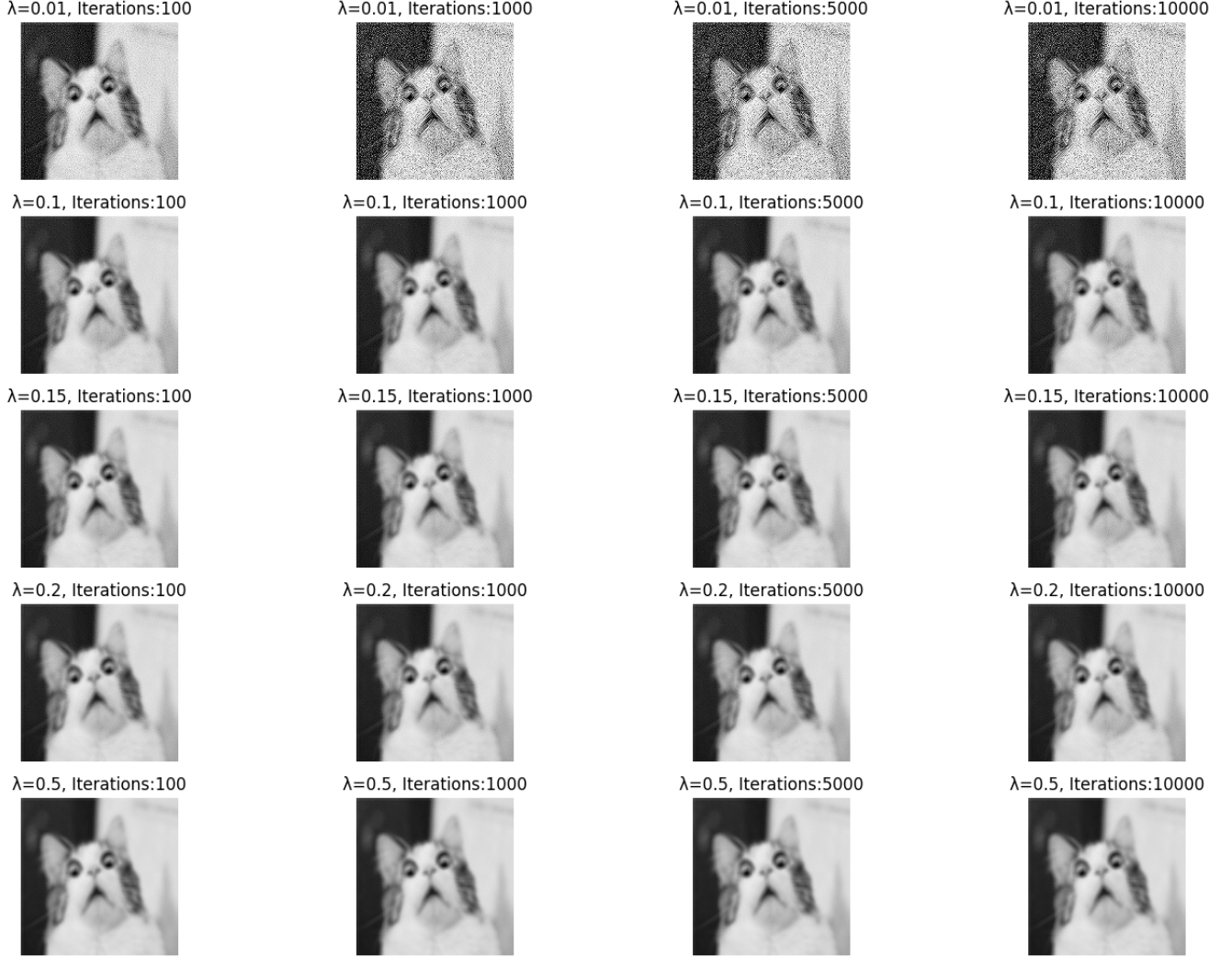


Figure 2: Comparison of the deblurred image by PD using different values for λ and different number of iterations

From the plot above, we make several observations. First, the optimal values for λ appear to fall within the range $[0.1, 0.2]$, with $\lambda = 0.15$ selected as it yields the closest resemblance to the original image. The number of iterations was picked to be 5000 as there seems to be a depreciation when comparing with 10^4 and it is clear that fewer iterations result in noticeably blurrier outcomes. Note that different values for σ (and τ analogously) were also checked, but $\sigma = \frac{1}{2\sqrt{2}}$ gives indeed the best results:

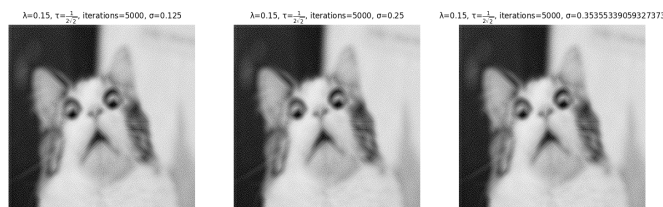


Figure 3: Comparison in between different σ 's

A.2 ADMM

In the case of ADMM we only try for three different values of λ and 3 different number of iterations as it is clear which combination yields the best results and changing the values further will not give us any new insights. Clearly, the only viable option is $\lambda = 0.01$ with 10 iterations.

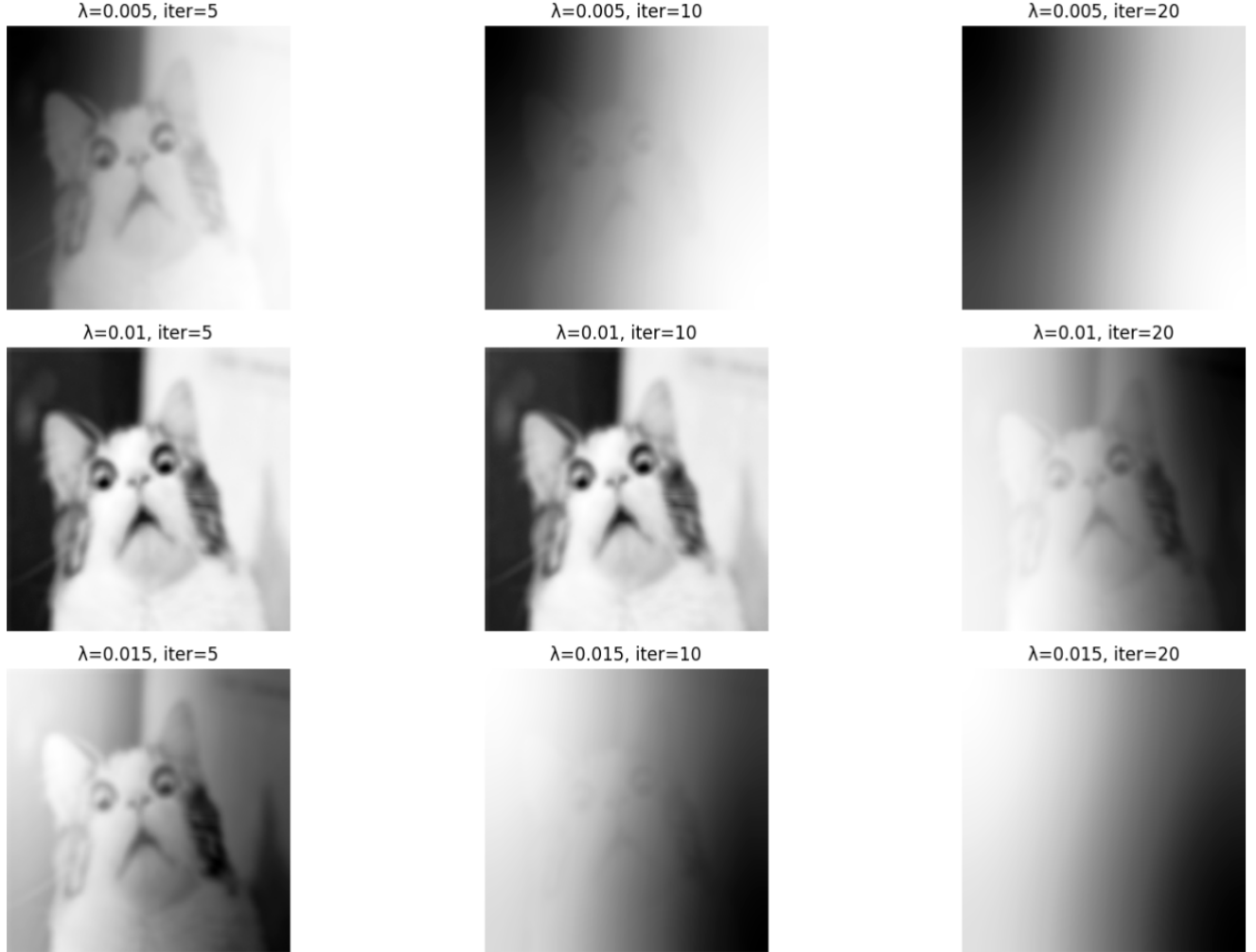


Figure 4: Comparison of the deblurred image by ADMM using different values for λ and different number of iterations

Below we show the possible options for ρ , where $\rho = 200$ seems most appropriate.

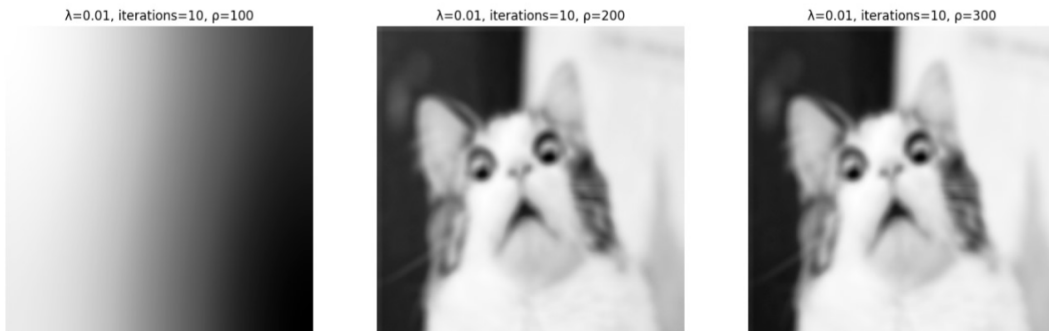


Figure 5: Comparison in between different ρ 's

B Additional Computations

B.1 Primal-Dual

In order to compute $\text{prox}_{\sigma g^*}$, we first compute $\text{prox}_{\sigma^{-1}g}$ and then apply *Moreau's Identity* given by

$$\text{prox}_{\sigma g^*}(y) = y - \sigma \text{prox}_{\sigma^{-1}g}(\sigma^{-1}y).$$

We have that $g(x) = \|x\|_1$ hence

$$\text{prox}_{\sigma^{-1}g}(y) = \text{argmin} \left\{ \sigma^{-1} \|x\|_1 + \frac{1}{2} \|x - y\|^2 \right\}$$

Note that this problem is summable thus we solve for

$$\text{prox}_{\sigma^{-1}g}(y)_i = \text{argmin} \left\{ \sigma^{-1} |x_i| + \frac{1}{2} (x_i - y_i)^2 \right\}$$

Denote $\varphi(x) = \sigma^{-1} |x_i| + \frac{1}{2} (x_i - y_i)^2$. In order to find a minimizer, differentiate with respect to x_i and equate to 0, by *Fermat's First Rule*. We distinguish in between three cases

1. Suppose $x_i > 0$. Then

$$0 = \frac{\partial \varphi}{\partial x_i} = \sigma^{-1} + x_i - y_i \iff x_i = y_i - \sigma^{-1}$$

Clearly, we must have $y_i > \sigma^{-1}$.

2. Assume $x_i < 0$ and compute the partial derivative

$$0 = \frac{\partial \varphi}{\partial x_i} = -\sigma^{-1} + x_i - y_i \iff x_i = y_i + \sigma^{-1}$$

thus $y_i < -\sigma^{-1}$.

3. Finally, if $x_i = 0$ we have that $0 = \frac{\partial \varphi}{\partial x_i}$ and hence we can take $|y_i| \leq \sigma^{-1}$.

The three cases then yield $\text{prox}_{\sigma^{-1}g}(y)_i = \text{sign}(y_i) \max(|y_i| - \sigma^{-1}, 0)$. By *Moreau's Identity* we have

$$\begin{aligned} \text{prox}_{\sigma g^*}(y)_i &= y_i - \sigma \text{sign} \left(\frac{y_i}{\sigma} \right) \max \left(\left| \frac{y_i}{\sigma} \right| - \sigma^{-1}, 0 \right) \\ &= y_i - \text{sign}(y_i) \max(|y_i| - 1, 0) \end{aligned}$$

Note that if $|y_i| > 1$, we have

$$\text{prox}_{\sigma g^*}(y)_i = \text{sign}(y_i) |y_i| - \text{sign}(y_i) |y_i| + \text{sign}(y_i) \cdot 1 = \text{sign}(y_i)$$

and for $|y_i| \leq 1, \max(|y_i| - 1, 0) = 0$ and hence

$$\text{prox}_{\sigma g^*}(y)_i = y_i.$$

Therefore, we conclude that

$$\text{prox}_{\sigma g^*}(y)_i = \begin{cases} y_i & \text{if } y_i \in [-1, 1] \\ \text{sign}(y_i) & \text{otherwise} . \end{cases}$$

Now, let us compute the proximity operator for τf . Again, we will make use of *Moreau's Identity*. To make our lives easier, we will take $f(x) = \frac{\lambda}{2} \|Rx - b\|^2$. In Homework 3 we computed the conjugate of this function for $\lambda = 1$. For other values of λ the computations are entirely similar hence we skip that part. We have $f^*(z) = \frac{1}{2\lambda} \|z\|^2 + b \cdot z$. Then *Moreau's Identity* gives

$$\begin{aligned} \text{prox}_{\tau^{-1}f^*}(y) = y - \tau^{-1} \text{prox}_{\tau f}(\tau y) &\iff \text{prox}_{\tau^{-1}f^*}(\tau^{-1}y) = \tau^{-1}y - \tau^{-1} \text{prox}_{\tau f}(y) \\ &\iff \text{prox}_{\tau f}(y) = y - \tau \text{prox}_{\tau^{-1}f^*}(\tau^{-1}y) \end{aligned}$$

Thus we are left to compute the proximity operator for $\tau^{-1}f^*$. That is, solve

$$\text{prox}_{\tau^{-1}f^*}(y) = \text{argmin} \left\{ \frac{1}{2\lambda\tau} \|x\|^2 + \frac{1}{\tau} x \cdot b + \frac{1}{2} \|x - y\|^2 \right\}$$

Denote $\psi(x) := \frac{1}{2\lambda\tau} \|x\|^2 + \frac{1}{\tau} x \cdot b + \frac{1}{2} \|x - y\|^2$. Then, we will find a minimizer using Fermat's First Rule. The gradient of ψ is given by

$$\nabla\psi(x) = \frac{1}{\tau\lambda}x + \frac{1}{\tau}b + x - y$$

and equating to 0 and solving for x gives

$$\begin{aligned} \left(\frac{1}{\tau\lambda} + 1 \right) x = y - \tau^{-1}b &\iff \frac{1 + \tau\lambda}{\tau\lambda} x = y - \tau^{-1}b \\ &\iff x = \frac{\tau\lambda}{1 + \tau\lambda} (y - \tau^{-1}b) \end{aligned}$$

Hence we have

$$\begin{aligned} \text{prox}_{\tau f}(y) &= y - \tau \text{prox}_{\tau^{-1}f^*}(\tau^{-1}y) \\ &= y - \tau \cdot \frac{\lambda}{1 + \tau\lambda} (y - b). \end{aligned}$$

B.2 ADMM

We write $\min_{\substack{x \in \mathbb{R}^{N_1}, \\ y \in \mathbb{R}^{N_2}}} \left\{ \frac{\lambda}{2} \|Rx - b\|^2 + \|y\|_1 \right\}$ subject to $Gx = y$. Let $f(x) = \frac{\lambda}{2} \|Rx - b\|^2$ and $g(y) = \|y\|_1$.

We need to compute

$$\begin{cases} x_{k+1} = \text{argmin}\{L_\rho(x, y_k, z_k) : x \in \mathbb{R}^{N_1}\}, \\ y_{k+1} = \text{argmin}\{L_\rho(x_{k+1}, y, z_k) : y \in \mathbb{R}^{N_2}\} \\ z_{k+1} = z_k + \rho(Gx_{k+1} - y_{k+1}). \end{cases} \iff \begin{cases} x_{k+1} = \text{argmin}\{f(x) + z_k^T(Gx - y_k) + \frac{\rho}{2}\|Gx - y_k\|^2\}, \\ y_{k+1} = \text{argmin}\{g(y) + z_k^T(Gx_{k+1} - y) + \frac{\rho}{2}\|Gx_{k+1} - y\|^2\} \\ z_{k+1} = z_k + \rho(Gx_{k+1} - y_{k+1}). \end{cases}$$

Taking $w = \frac{z}{\rho}$ and noting that

$$\begin{aligned} \frac{\rho}{2} \|Gx - y + w\|^2 &= \frac{\rho}{2} (Gx - y + w)^T (Gx - y + w) \\ &= \frac{\rho}{2} (x^T G^T Gx - x^T G^T y + x^T G^T w - y^T Gx + y^T y - y^T w + w^T Gx - w^T y + w^T w) \\ &= \frac{\rho}{2} (x^T G^T Gx - 2x^T G^T y + y^T y) + \rho w^T Gx - \rho w^T y + \frac{\rho}{2} w^T w \\ &= \frac{\rho}{2} \|Gx - y\|^2 + z^T (Gx - y) + \frac{1}{2\rho} z^T z, \end{aligned}$$

the system can be rewritten as follows

$$\begin{cases} x_{k+1} = \operatorname{argmin} \left\{ \frac{\lambda}{2} \|Rx - b\|^2 + \frac{\rho}{2} \left\| Gx - y_k + \frac{z_k}{\rho} \right\|^2 \right\} \\ y_{k+1} = \operatorname{argmin} \left\{ \|y\|_1 + \frac{\rho}{2} \left\| y - Gx_{k+1} - \frac{z_k}{\rho} \right\|^2 \right\} \\ z_{k+1} = z_k + \rho(Gx_{k+1} - y_{k+1}) \end{cases}$$

Now, x_{k+1} minimizes $\varphi(x) = \frac{\lambda}{2} \|Rx - b\|^2 + \frac{\rho}{2} \left\| Gx - y_k + \frac{z_k}{\rho} \right\|^2$. From example **2.29** in the Lecture Notes and the first computational exercise we know that $\nabla h(x) = A^T(Ax - b)$, where $h(x) = \|Ax - b\|^2$ thus, it follows immediately that

$$0 = \lambda R^T(Rx_{k+1} - b) + \rho G^T(Gx_{k+1} - y_k + \frac{z_k}{\rho}) = \lambda R^T Rx_{k+1} - \lambda R^T b + \rho G^T Gx_{k+1} - G^T(\rho y_k - z_k)$$

hence,

$$(\lambda R^T R + \rho G^T G)x_{k+1} = \lambda R^T b + G^T(\rho y_k - z_k) \iff x_{k+1} = (\lambda R^T R + \rho G^T G)^{-1}(\lambda R^T b + G^T(\rho y_k - z_k))$$

where we have used *Fermat's First Rule* to find the minimizer. Next, y_{k+1} minimizes $\psi(y) = \|y\|_1 + \frac{\rho}{2} \left\| y - Gx_{k+1} - \frac{z_k}{\rho} \right\|^2$ and so y_{k+1} minimizes $\frac{1}{\rho}\psi(y)$, that is,

$$\begin{aligned} y_{k+1} &= \operatorname{argmin} \left\{ \frac{1}{\rho} \|y\|_1 + \frac{1}{2} \left\| y - Gx_{k+1} - \frac{z_k}{\rho} \right\|^2 \right\} \\ &= \operatorname{argmin} \left\{ \rho^{-1} g(y) + \frac{1}{2} \left\| y - Gx_{k+1} - \frac{z_k}{\rho} \right\|^2 \right\} \\ &= \operatorname{prox}_{\rho^{-1}g} \left(Gx_{k+1} + \frac{z_k}{\rho} \right). \end{aligned}$$

As seen in the computations for the Primal-Dual method, we obtain

$$\begin{aligned} \operatorname{prox}_{\rho^{-1}g} \left(Gx_{k+1} + \frac{z_k}{\rho} \right)_i &= \begin{cases} \left(Gx_{k+1} + \frac{z_k}{\rho} \right)_i - \frac{1}{\rho} & \text{if } \left(Gx_{k+1} + \frac{z_k}{\rho} \right)_i > \frac{1}{\rho} \\ \left(Gx_{k+1} + \frac{z_k}{\rho} \right)_i + \frac{1}{\rho} & \text{if } \left(Gx_{k+1} + \frac{z_k}{\rho} \right)_i < -\frac{1}{\rho} \\ 0 & \text{otherwise} \end{cases} \\ &= \operatorname{sign} \left(Gx_{k+1} + \frac{z_k}{\rho} \right)_i \max \left(\left| Gx_{k+1} + \frac{z_k}{\rho} \right|_i - \frac{1}{\rho}, 0 \right). \end{aligned}$$

Hence, $y_{k+1} = \operatorname{sign} \left(Gx_{k+1} + \frac{z_k}{\rho} \right) \max \left(\left| Gx_{k+1} + \frac{z_k}{\rho} \right| - \frac{1}{\rho}, 0 \right)$. Finally, z_{k+1} is simply $z_{k+1} = z_k + \rho(Gx_{k+1} - y_{k+1})$.

C Code

C.1 Primal-Dual

This function implements the Primal-Dual Method using the computations in Appendix B.1.

```
1 def primal_dual(X_blur, R, grad, grad_T, lambda_, tau, sigma, num_iterations):
2     # Proximal operator for f
3     def prox_f_star(x, tau, lambda_, X_blur):
4         return ((tau * lambda_ / (tau * lambda_ + 1)) * (x - (1/tau) * X_blur))
5
6     def prox_f(x, tau, lambda_, X_blur):
7         return x - tau * prox_f_star(x/tau, tau, lambda_, X_blur)
8
9     # Proximal operator for g*
10    def prox_g_star(y):
11        return np.clip(y, -1, 1)
12
13    # Initialize primal and dual variables
14    X = X_blur.copy()
15    Y = grad(X).copy()
16
17
18    # Primal-Dual Iteration
19    for k in range(num_iterations):
20        # Update primal variable X
21        X_prev = X
22        X = prox_f(X - tau * grad_T(Y), tau, lambda_, X_blur)
23
24        # Update dual variable Y
25        Y_prev = Y
26        Y = prox_g_star(Y_prev + sigma * grad(2 * X - X_prev))
27
28    return X
```

Now, in order to find the best values for λ and the number of iterations, the following code was implemented.

```
1 # Parameters to test
2 lambdas = [0.01, 0.05, 0.1, 0.15, 0.2, 0.25]
3 maxits = [10**2, 10**3, 5*10**3, 10**4]
4 # Creating the figure to hold subplots (len(lambdas) rows, len(taus) columns)
5 fig, axes = plt.subplots(len(lambdas), len(maxits), figsize=(15, 10))
6
7 for i, lam in enumerate(lambdas):
8     for j, maxit in enumerate(maxits):
9         #print(f"Testing lambda={lam}, tau={tau}, sigma={sigma}")
10        X_deblur = primal_dual(X_blur, R, grad, grad_T, lambda_=lam, tau=1/(2*np.sqrt(2)),
11                               sigma=1/(2*np.sqrt(2)), num_iterations=maxit)
12
13        # Select the appropriate subplot
14        ax = axes[i, j] # Use the i-th row and j-th column for this subplot
15        ax.imshow(X_deblur, cmap=cm.Greys_r, vmin=0, vmax=255)
16        ax.set_title(f'l={lam}, Iterations:{maxit}')
17        ax.axis('off')
18
19 plt.show()
```

Additionally, we check for different values of σ (for different τ 's the code is entirely similar).

```
1 # Parameters to test
2 sigmas = [1/8, 1/4, 1/(2*np.sqrt(2))]
```

```

3 lambda_ = 0.15
4
5 # Creating the figure to hold subplots (len(lambdas) rows, len(taus) columns)
6 fig, axes = plt.subplots(1, len(sigmals), figsize=(15, 5))
7
8 for j, sigma in enumerate(sigmals):
9     #print(f"Testing lambda={lam}, tau={tau}, sigma={sigma}")
10    X_deblur = primal_dual(X_blur, R, grad, grad_T, lambda_ = 0.15, tau=1/(2*np.sqrt(2)), sigma=
        sigma, num_iterations=5*10**3)
11
12    # Select the appropriate subplot
13    ax = axes[j] # Use the i-th row and j-th column for this subplot
14    ax.imshow(X_deblur, cmap=cm.Greys_r, vmin=0, vmax=255)
15    ax.set_title(f"l={0.15}, t=r'\frac{1}{2\sqrt{2}}$'f", iterations={5000}, s={sigma}")
16    ax.axis('off')
17
18 # Adjust layout for better spacing
19 plt.tight_layout()
20 plt.subplots_adjust(top=0.85)
21 plt.show()

```

C.2 ADMM

The following function implements ADMM using the results in Appendix B.2.

```

1 from scipy.sparse.linalg import cg, LinearOperator
2
3 # ADMM function
4 def admm_deblur(X_blur, R, grad, grad_T, lambda_, rho, num_iter):
5     # Define operator A
6     def A_operator(X, R, rho, grad, grad_T):
7         RTR_term = lambda_ * R(R(X))
8         grad_term = rho * grad_T(grad(X))
9         A_X = RTR_term + grad_term
10        return A_X.ravel()
11
12    def soft_thresholding(Y, tau):
13        return np.sign(Y) * np.maximum(np.abs(Y) - tau, 0)
14    # Initialization
15    X = X_blur
16    Y = grad(X)
17    Z = Y
18
19    for k in range(num_iter):
20        #print(k)
21        # Solve for X^{k+1} using Conjugate Gradient
22        b = lambda_ * R(X_blur) + grad_T((rho * Y) - Z)
23        # Pass necessary arguments to A_operator
24        A = LinearOperator((np.prod(X.shape), np.prod(X.shape)), matvec=lambda x: A_operator(x.reshape(X
        .shape), R, rho, grad, grad_T))
25
26        X_solution, info = cg(A, b.ravel(), maxiter=10000)
27
28        if info != 0:
29            print(f"CG did not converge, info: {info}")
30
31        X = X_solution.reshape(X.shape)
32
33        # Update Y^{k+1} (Soft-thresholding)

```

```

34     Y = soft_thresholding(grad(X) + Z/rho, 1 / rho)
35
36     # Update  $Z^{k+1}$ 
37     Z += rho * (grad(X) - Y)
38
39     return X

```

In order to find the optimal λ and number of iterations, the following code was implemented.

```

1 # Define parameter ranges
2 lambdas = [0.005, 0.01, 0.015]
3 iters = [5, 10, 20]
4
5 # Plotting setup
6 fig, axes = plt.subplots(len(lambdas), len(iters), figsize=(15, 10))
7 #fig.suptitle('Deblurred Images for Different Lambdas and Rhos', fontsize=16)
8
9 for i, lam in enumerate(lambdas):
10     for j, iter in enumerate(iters):
11         print(f"Testing lambda={lam}, iter={iter}")
12         X_deblurred = admm_deblur(X_blur, R, grad, grad_T, lambda_=lam, rho=200, num_iter=iter)
13         ax = axes[i, j]
14         ax.imshow(X_deblurred, cmap=cm.Greys_r)
15         ax.set_title(f'l={lam}, iter={iter}')
16         ax.axis('off')
17
18 plt.tight_layout()
19 plt.subplots_adjust(top=0.9)
20 plt.show()

```

Moreover, we also try for different ρ 's. The code is shown below.

```

1 # Define parameter ranges
2 lambda_ = 0.01
3 rhos = [100, 200, 300]
4
5 # Plotting setup
6 fig, axes = plt.subplots(1, len(rhos), figsize=(15, 5))
7
8
9 for j, rho in enumerate(rhos):
10     print(f"Testing lambda={lambda_}, rho={rho}")
11     X_deblurred = admm_deblur(X_blur, R, grad, grad_T, lambda_=lambda_, rho=rho, num_iter=10)
12     ax = axes[j]
13     ax.imshow(X_deblurred, cmap=cm.Greys_r)
14     ax.set_title(f'l={lambda_}, iterations={10}, r={rho}')
15     ax.axis('off')
16
17 plt.tight_layout()
18 plt.subplots_adjust(top=0.85)
19 plt.show()

```

C.3 Final Comparison

Finally, the best found values for λ and the number of iterations chosen from the previous plots are used for the comparison between the original image, the blurred image, the deblurred image by PD and the deblurred image by ADMM.

```

1 import matplotlib.cm as cm
2
3 # Plotting the results

```

```

4 plt.figure(figsize=(16, 4)) # Adjust the figure size for better visibility
5
6 # Original Image
7 plt.subplot(1, 4, 1)
8 plt.imshow(X_ref, cmap=cm.Greys_r, vmin=0, vmax=255)
9 plt.title("Original Image")
10 plt.axis('off')
11
12 # Blurred Image
13 plt.subplot(1, 4, 2)
14 plt.imshow(X_blur, cmap=cm.Greys_r, vmin=0, vmax=255)
15 plt.title("Blurred Image")
16 plt.axis('off')
17
18 # Deblurred Image (Primal-Dual)
19 plt.subplot(1, 4, 3)
20 plt.imshow(X_deblur_PD, cmap=cm.Greys_r, vmin=0, vmax=255)
21 plt.title("Deblurred Image (Primal-Dual)")
22 plt.axis('off')
23 plt.text(0.5, -0.05, r'$t=s = \frac{1}{2\sqrt{2}}$', l=0.15 and 5000 iterations', ha='center', va='top',
        fontsize=10, transform=plt.gca().transAxes)
24
25 # Deblurred Image (ADMM)
26 plt.subplot(1, 4, 4)
27 plt.imshow(X_deblur_ADMM, cmap=cm.Greys_r)
28 plt.title("Deblurred Image (ADMM)")
29 plt.axis('off')
30 plt.text(0.5, -0.05, 's=200, l=0.01 and 10 iterations', ha='center', va='top', fontsize=10, transform=
        plt.gca().transAxes)
31
32 plt.tight_layout()
33 plt.show()

```